

A Multiscale, Laterally Connected Predictive-Coding Cap for Continual Learning on ePC Transformers

Architecture hypothesis, comparison with backpropagation, and coding-agent implementation guide

ZeroBot / ProtoCosmoBot

Synthesis of the Bot Philosophy discussion with Ben Goertzel and ProtoMegaBot

30 July 2026

Abstract

This note proposes a small recurrent predictive-coding (PC) “cortical island” above an error-predictive-coding (ePC) transformer. Three cap levels combine hierarchical prediction, bottom-up error, sparse lateral attractor dynamics, multiple learning timescales, and recurrence-gated consolidation. Novel input is first quarantined in a fast episodic level; only repeated, coherent novelty is promoted into slower contextual and schema levels and allowed to modulate base-network plasticity. The main claim is conditional and falsifiable: if an ePC and a backpropagation (BP) transformer have matched base-task quality, an ePC base should provide a cleaner substrate for this cap because prediction errors, precision, top-down replay, settling dynamics, and local plasticity modulation share one computational idiom. The report gives interfaces, update equations, failure controls, ablations, and explicitly speculative starting configurations at GPT-2, GPT-3, GLM-5.2, and Kimi-K3 scale.

Epistemic status. This is a design hypothesis, not an established continual-learning result. Published model dimensions are labeled as facts; cap sizes and learning rates are engineering guesses to seed sweeps. In particular, “one unified energy” requires compatible objectives and reciprocal parameterizations; merely joining two modules called PC does not guarantee a global Lyapunov function or convergence.

1 Operational objective

Let a base transformer produce a residual representation $z_t \in \mathbb{R}^d$ for a token, segment, or pooled event at time t . We want a cap that learns online while reducing destructive interference, without requiring long globally synchronized retraining cycles. It should:

1. recognize familiar instances, novel combinations, and genuinely novel regimes;
2. form sparse, reusable context attractors from repeated experience;
3. preserve uncertain novelty in a fast buffer before altering consolidated knowledge;
4. send the base a context vector, uncertainty, and spatially selective plasticity gates;
5. support latent generative replay of old contexts;
6. learn with bounded local updates and short, optionally asynchronous settling.

The cap is not expected to solve continual learning alone. Its value must be tested against matched recurrent memory, replay, regularization, and parameter-isolation controls.

2 Architecture: a three-level recurrent PC cap

2.1 State and connectivity

Use three cap states y_t^1, y_t^2, y_t^3 :

$$z_t \leftrightarrow y_t^1 \leftrightarrow y_t^2 \leftrightarrow y_t^3, \quad y_t^\ell \leftrightarrow L_\ell y_t^\ell, \quad y_{t-1}^\ell \rightarrow y_t^\ell.$$

The arrows between levels are reciprocal but need not be transposes. Each level contains:

- a top-down generator g_ℓ and bottom-up error encoder q_ℓ ;
- fixed or slowly learned lateral inhibition for sparse competition;

- learned block-sparse lateral excitation for associative attractors;
- a recurrent trace for temporal persistence;
- a precision head π^ℓ and consolidation statistic c^ℓ .

Level	Primary content	Plasticity	Characteristic role
y^1 episodic	local motifs and recent residual patterns	fast	capture/quarantine novelty; sparse instance code
y^2 contextual	recurring coalitions and situations	medium	context attractors; identify novel combinations
y^3 schema	regimes, tasks, domains	slow	consolidated prior; replay seed; base-gating policy

2.2 Local energy and settling

One implementable energy for a microbatch/window is

$$\mathcal{E} = \sum_{\ell=0}^2 \frac{1}{2} \left\| \Pi_\ell^{1/2} (y^\ell - g_{\ell+1}(y^{\ell+1})) \right\|^2 + \sum_{\ell=1}^3 \lambda_\ell \Phi_\ell(y^\ell; L_\ell) + \sum_{\ell=1}^3 \rho_\ell \|y^\ell - A_\ell y_{t-1}^\ell\|^2,$$

where $y^0 = z$, $\Pi_\ell = \text{diag}(\pi^\ell)$ is bounded precision, Φ_ℓ combines sparse competition and lateral consistency, and A_ℓ is a temporal trace. During cap inference, freeze weights and perform K_ℓ damped local state steps:

$$y^\ell \leftarrow y^\ell - \alpha_\ell \text{clip}(\nabla_{y^\ell} \mathcal{E}, c_y).$$

Do not assume monotonic energy descent: test it. Backtrack or reduce α_ℓ when $\mathcal{E}_{k+1} > \mathcal{E}_k + \epsilon_E$. For streaming use, layers may update asynchronously using versioned state snapshots; stale reads must be bounded and measured.

2.3 Lateral connections: two distinct functions

The phrase “lateral connection” should not conceal two mechanisms:

1. **competition/normalization:** fixed inhibitory or divisive dynamics produce sparse codes and prevent every attractor from co-activating;
2. **association/precision:** learned block-sparse excitation represents co-occurring coalitions and contributes to confidence/precision.

Precision must be bounded, e.g.

$$\pi^\ell = \pi_{\min} + (\pi_{\max} - \pi_{\min}) \sigma(P_\ell y^\ell + b_\ell),$$

because unconstrained learned precision can “explain away” errors by driving gain to zero or infinity. Precision is not identical to attractor depth; the proposed relationship is a hypothesis requiring calibration against held-out prediction reliability.

3 Continual-learning rule

3.1 Do not equate surprise with permission to overwrite

Raw high error may be coherent novelty, an outlier, corruption, or a base-representation shift. Therefore use a two-stage gate:

1. **capture:** large lower-level error creates or updates a fast y^1 trace while base plasticity is conservative;
2. **promotion:** recurrence across independent windows, predictive improvement, and agreement at y^2/y^3 increase consolidation confidence; only then may the cap selectively open base plasticity.

For event u , a simple promotion score is

$$s_u = \sigma(a \log(1 + n_u) + b \Delta \mathcal{E}_u - c \text{Var}(e_u) - d o_u - \tau),$$

where n_u is recurrence count, $\Delta \mathcal{E}_u$ is predictive improvement, and o_u is an outlier score. Require recurrence in separated windows, not repeated tokens within one sequence.

3.2 Multi-level error pattern and action

Observed cap error	Interpretation (tentative)	Plasticity response
e^1 low	familiar instance	near-zero base update; optional slow rehearsal
e^1 high; $e^{2,3}$ low	new instance of known context/schema	moderate update near readout or selected upper base blocks
e^1 low; $e^{2,3}$ high	familiar features in unfamiliar composition	update attention/composition modules; protect feature modules
all levels high, low outlier score, recurrent	coherent new regime	recruit cap capacity, then progressively open selected base modules
all levels high, nonrecurrent or high outlier score	noise/corruption/one-off	quarantine; no broad base update

The cap emits three outputs:

$$c_t = D_c y_t^1, \quad u_t = D_u(e_t^1, e_t^2, e_t^3, \pi_t), \quad m_{b,t} \in [0, 1] \quad (b = 1, \dots, B),$$

where c_t is a residual context, u_t is calibrated uncertainty, and $m_{b,t}$ gates the learning rule of base block b . Initialize D_c at zero so adding the cap is functionally identity-preserving.

3.3 Local weight updates and consolidation

For reciprocal prediction matrix W_ℓ , a PC-local candidate update is

$$\Delta W_\ell = \eta_\ell s_t m_{\ell,t} \text{clip}((\Pi_{\ell-1} e^{\ell-1})(y^\ell)^\top, c_W) - \eta_\ell \lambda_W W_\ell.$$

Use eligibility traces if updates are delayed. Keep three learning-rate bands with $\eta_1 : \eta_2 : \eta_3$ initially around 100 : 10 : 1. This ratio is a sweep seed, not a law. Consolidation may also maintain an exponential moving average (EMA) teacher for each cap level. Replay begins at sampled y^3 schemas, settles laterally at each level, generates pseudo- z , and interleaves it with new windows. Reject replay samples whose energy, precision, or diversity lies outside the training envelope.

4 Why an ePC transformer is the cleaner base

Hypothesis. Conditional on comparable base-task quality and a correctly implemented ePC learner, the cap should integrate more naturally with ePC than with a conventional BP transformer. The reasons are structural, but none substitutes for a controlled comparison.

1. **Signal compatibility.** Both modules expose layerwise predictions, errors, precision, and reciprocal influence. The interface can be one more PC boundary rather than an auxiliary density model glued to a feed-forward representation.
2. **Surgical plasticity modulation.** An ePC local update can directly multiply its synapse/block error term by $m_{b,t}$. BP can also be masked or modulated, so “BP cannot do this” would be false; the narrower claim is that BP’s downstream credit assignment and optimizer state make the intervention more globally entangled and less PC-native.
3. **Native top-down influence and latent replay.** A reciprocal ePC base already contains a path through which cap hypotheses can constrain lower activities and reconstruct old latent regimes. A standard causal transformer needs adapters, prefix/context injection, an auxiliary decoder, or input-space replay.
4. **Settling trajectory as evidence.** Residual error, convergence time, energy slope, and oscillation supply familiarity/uncertainty features beyond a single forward state. These signals are useful only if calibrated; slow settling can also simply indicate poor conditioning.
5. **Potential common objective.** If the cap and ePC base share explicitly compatible local energies and reciprocal mappings, joint inference may minimize a declared global energy. This enables stability analysis. It is not automatic: asymmetric feedback, stale asynchronous states, precision learning, and separate task losses can break the claim.

6. **One hierarchy with a timescale gradient.** The cap/base boundary can become an engineering seam within a deeper PC hierarchy: fast task-sensitive lower levels and slow context/schema upper levels. Continual learning may then emerge from sparse recruitment, precision, replay, and timescale separation rather than a single anti-forgetting penalty.

4.1 Honest BP counterargument

BP transformers have mature kernels, pretrained weights, scaling evidence, and effective continual-learning tools: replay, adapters, routing, regularization, gradient projection, and parameter isolation. BP permits per-block gradient masks, so the comparison must not use a straw man. ePC additionally pays settling latency and may suffer equilibrium, stability, and credit-approximation errors. The defensible claim is therefore:

At matched base quality, parameter budget, replay budget, and wall-clock, does the PC cap exploit an ePC base more effectively?

5 Implementation contract for coding agents

5.1 Module boundaries

```
BaseProbe:
  observe(hidden_by_block, error_by_block, settle_trace) -> BaseObservation
CapStateStore:
  load(stream_id, version) / compare_and_swap(stream_id, state)
PCCap:
  settle(observation, prior_state, K, masks) -> CapInference
PromotionGate:
  score(cap_inference, recurrence_stats, outlier_stats) -> Promotion
PlasticityRouter:
  route(promotion, cap_errors, base_errors) -> block_gates
LocalUpdater:
  accumulate_eligibility(...) / apply(gates, learning_rates)
ReplaySampler:
  sample_schema() / decode_and_validate() -> pseudo_observation
Telemetry:
  record(errors, energy, precision, gates, promotions, interference)
```

Keep these interfaces replaceable. A first implementation may use ordinary autograd to verify equations, but it must label that path `reference_autograd`; it must not be reported as a local-learning implementation.

5.2 Numerical invariants

1. Adding a zero-initialized cap changes base logits by at most declared tolerance.
2. Base and cap weights remain frozen during state settling.
3. Error, energy, and precision accumulation use FP32 even when activations are BF16.
4. No dropout, mutable KV cache, or nondeterministic routing changes within a paired settling/replay comparison.
5. Every state update is finite; precision stays in $[\pi_{\min}, \pi_{\max}]$.
6. Accepted settle steps satisfy the energy policy or record a backtrack/failure.
7. Local updates have explicit stop-gradient boundaries and unit tests against analytic linear cases.
8. Asynchronous state writes use version checks; rejected stale writes are counted.
9. MoE routing metadata is recorded; cap gates must not silently change expert selection during a matched comparison unless routing modulation is the tested treatment.

5.3 Build sequence

1. Implement a linear three-level cap and verify analytic gradients, energy descent, identity initialization, and local-update direction.

2. Add fixed lateral inhibition; test target sparsity and absence of dead units.
3. Add learned block-sparse excitation with spectral-radius constraint.
4. Add temporal traces and recurrence-based promotion; test outlier quarantine.
5. Connect read-only to a frozen small ePC transformer; calibrate errors and precision.
6. Enable cap-only learning, then gated adapters/LoRA, then selected base blocks.
7. Add top-down replay only after reconstruction and diversity gates pass.

6 Quantitative starting hypotheses

6.1 What scale should track

Cap width should track the base residual-stream width d and diversity of online contexts, not total model parameters. This is crucial for sparse MoE systems: total expert parameters do not all appear in one token’s residual stream. Start with cap widths c_1, c_2, c_3 , factorized reciprocal maps of ranks r_1, r_2, r_3 , lateral blocks of 64 units, and a small prototype bank. Parameter estimates below include factorized reciprocal maps plus one 64-wide lateral and one lighter temporal operator per level; they exclude optimizer state, replay buffers, and optional decoders.

Base case	Published base configuration used	Proposed cap widths (c_1, c_2, c_3)	Map ranks (r_1, r_2, r_3)	Core cap estimate and starting regime
GPT-2 small	$d = 768$, 12 layers, 12 heads, 124M class	(512, 256, 128)	(128, 64, 32)	$\sim 0.6\text{M}$ parameters. $K = (3, 3, 5)$; 64–256 context prototypes; cap LR $10^{-4}, 10^{-5}, 10^{-6}$; base initially frozen, then top 2–4 blocks via LoRA.
GPT-3 175B	$d = 12,288$, 96 layers, 96 heads	(4096, 2048, 1024)	(512, 256, 128)	$\sim 22\text{M}$ core. $K = (2, 3, 4)$; 1K–4K prototypes; cap LR $3 \times 10^{-5}, 3 \times 10^{-6}, 3 \times 10^{-7}$; gate 8–12 block groups rather than 96 independent scalars initially.
GLM-5.2	$d = 6144$, 78 layers; 256 routed experts, top-8; first 3 MLP layers dense	(3072, 1536, 768)	(384, 192, 96)	$\sim 10\text{M}$ core. $K = (2, 3, 4)$; 1K–2K prototypes. Separate gates for attention, shared expert, routed experts, and router; freeze router first.
Kimi-K3	text $d = 7168$, 93 layers; 896 experts, top-16 plus 2 shared; hybrid KDA/full-attention	(3584, 1792, 896)	(448, 224, 112)	$\sim 14\text{M}$ core. $K = (2, 3, 4)$; 2K–4K prototypes. Gate KDA, full-attention, shared experts, routed experts, and router separately; begin text-only and freeze router.

Published fact. The GLM-5.2 values above come from the official public configuration: hidden size 6144, 78 layers, 256 routed experts, 8 selected per token, one shared expert, and three initial dense MLP layers. **Published fact.** The Kimi-K3 values come from its official public configuration/model card: text hidden size 7168, 93 layers, 896 experts, 16 selected per token, two shared experts, and a hybrid of KDA and full-attention layers. **Engineering guess.** All cap dimensions, ranks, prototype counts, settling steps, and learning rates are guesses.

6.2 Why not scale the cap linearly with total parameters?

At GPT-3 and MoE scale, dense $d \times c_1$ reciprocal maps are unnecessarily large. Factorize them and share cap computation across block groups. The cap represents slow contexts over the residual stream; it need not mirror all base layers or experts. For MoE, start with residual/context gating and router readout only. Allow router plasticity only after showing that context gating improves retention without routing collapse or expert starvation.

6.3 Memory and cadence

- Run cap inference on pooled 8–64-token events rather than every token in the first prototype. Compare mean, attention, and learned event pooling.
- Maintain fast traces for 32–256 events, prototypes for 64–4096 contexts depending on scale, and an explicit eviction/merge policy.
- Try one cap update per 8–32 base events; one y^2 consolidation per 8–32 y^1 promotions; one y^3 update per 8–32 y^2 promotions.
- Limit replay initially to 10–25% of online events and match wall-clock in controls.

7 Tuning plan and diagnostics

7.1 Tune in this order

1. **State dynamics:** α_ℓ , damping, normalization, spectral radius, K_ℓ , energy/backtracking rate.
2. **Code quality:** sparsity target (start 5–15%), dead-unit fraction, attractor recovery under corruption, context separation.
3. **Precision calibration:** reliability diagrams and expected calibration error for predicted reconstruction/prediction quality.
4. **Promotion:** recurrence threshold, separated-window requirement, outlier gate, false promotion/false quarantine.
5. **Plasticity routing:** start cap-only, then adapters, then base block groups.
6. **Replay:** only after pseudo-latent fidelity, diversity, and no-collapse tests.

7.2 Required telemetry

Log per level: normalized error, energy before/after each settle step, precision quantiles, state norm, sparsity, attractor assignment entropy, recurrence counts, promotions, merges, evictions, and update norms. Log per base block: gate, local error, update norm, representation drift, router/expert statistics for MoE, and settling cost. Report throughput, latency, memory, and energy/backtrack failures. Averages alone are insufficient; retain tails and per-context breakdowns.

7.3 Failure signatures

Signature	Likely cause	First response
all contexts merge	weak inhibition, high lateral radius, overly fast y^2/y^3 learning	increase competition; lower LR/radius; add orthogonality/diversity control
every event becomes new	miscalibrated precision, base drift, threshold too low	normalize errors by context; slow adapter; raise separated recurrence threshold
no promotion	cap too rigid or precision too low	lower threshold; increase y^1 capacity; inspect predictive-improvement term
energy oscillates	step too large, asymmetric maps, stale asynchronous state	damp/backtrack; bound delay; test reciprocal parameterization
old tasks retained, new task not learned	gates too conservative	open adapters/upper blocks; reduce consolidation delay
new task learned, old tasks collapse	outlier/surprise directly opens base	enforce quarantine; increase replay or parameter isolation
MoE expert starvation	cap/router feedback collapse	freeze router; entropy/load-balance gate; separate shared/routed-expert modulation

8 Evaluation and falsification

Use sequential regimes with controlled overlap: permutations, new compositions of old features, coherent new domains, and injected outliers. Report:

average accuracy/perplexity, backward transfer, forward transfer, worst-case forgetting, adaptation area-under-curve.

Also measure context identification, promotion precision/recall, error calibration, replay coverage, base representation drift, and wall-clock/energy cost.

Minimum matched arms:

1. ePC base alone;
2. ePC + one-layer novelty cap;
3. ePC + hierarchical cap without lateral excitation;
4. ePC + full cap without replay;
5. ePC + full cap with replay;
6. BP + same-cap interface via adapters and masked gradients;
7. BP + matched recurrent memory;
8. BP + strong replay/regularization or parameter-isolation baseline.

Match trainable parameters, replay bytes, online examples, optimizer state, and wall-clock as separate comparisons. A parameter-matched win that disappears under wall-clock matching is not sufficient.

Central falsifier. Reject the strong architectural hypothesis if, across multiple regime sequences and seeds, the ePC+cap fails to reduce forgetting/adaptation tradeoff relative to the strongest matched BP baseline, or if any gain is explained by extra replay, parameters, settling compute, or privileged task-boundary information.

9 Recommended first experiment

Start at GPT-2-small or smaller, not at GPT-3/MoE scale:

1. frozen pretrained base; train only the cap to predict pooled top-layer residuals;
2. three synthetic stream regimes plus outlier bursts;
3. compare one-layer, three-layer no-lateral, and full block-lateral cap;
4. pass gates for energy behavior, context recovery, precision calibration, and outlier quarantine;
5. then permit zero-initialized context injection and LoRA on the top four blocks;
6. finally compare ePC and BP bases under matched parameter and wall-clock budgets.

Only after the small model establishes a stable mechanism should the cap be ported to large dense or MoE bases. At MoE scale, router modulation is a separate experiment, not a default.

10 Sources and provenance

- Bot Philosophy Telegram discussion, messages 15198–15227, 30 July 2026: initial episodic cap, multilayer/lateral extension, and ePC-vs-BP rationale.
- OpenAI, *Language Models are Unsupervised Multitask Learners* (2019), GPT-2 model configurations: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- Brown et al., *Language Models are Few-Shot Learners* (2020), GPT-3 175B configuration: <https://arxiv.org/abs/2005.14165>
- Z.ai official GLM-5.2 configuration (retrieved 30 July 2026): <https://huggingface.co/zai-org/GLM-5.2/blob/main/config.json>
- Moonshot AI official Kimi-K3 configuration/model card (retrieved 30 July 2026): <https://huggingface.co/moonshotai/Kimi-K3/blob/main/config.json>, <https://github.com/MoonshotAI/Kimi-K3>

Bottom line. Build the cap as a small, slow, recurrent PC hierarchy that first captures and tests novelty, then consolidates it. The ePC advantage is not mystical: it is the possibility of sharing error, precision, top-down inference, and locally gated plasticity across one declared dynamical system. Whether that coherence beats mature BP continual-learning methods is an empirical question, and the implementation should be organized to make a negative result as informative as a positive one.